

Security Audit Report — OWASP Juice Shop

Team Members:

Syed Abdul Rehman, Dawood Sheikh, Syed Irtiza Ali

Course: Cyber Security and Ethical Hacking — PITP Phase II (GR-XI)

Deliverable: Modern Web App Security Audit

Objective

Perform a white-box security audit against the OWASP Juice Shop running locally in Docker. Find, exploit, and document five distinct vulnerabilities across at least three OWASP Top 10 categories. Use tools: Nmap, Burp Suite, Browser, and Docker (Kali Linux environment).

Setup & Access (How this was run)

1. Run Juice Shop locally in Kali Linux (Docker):

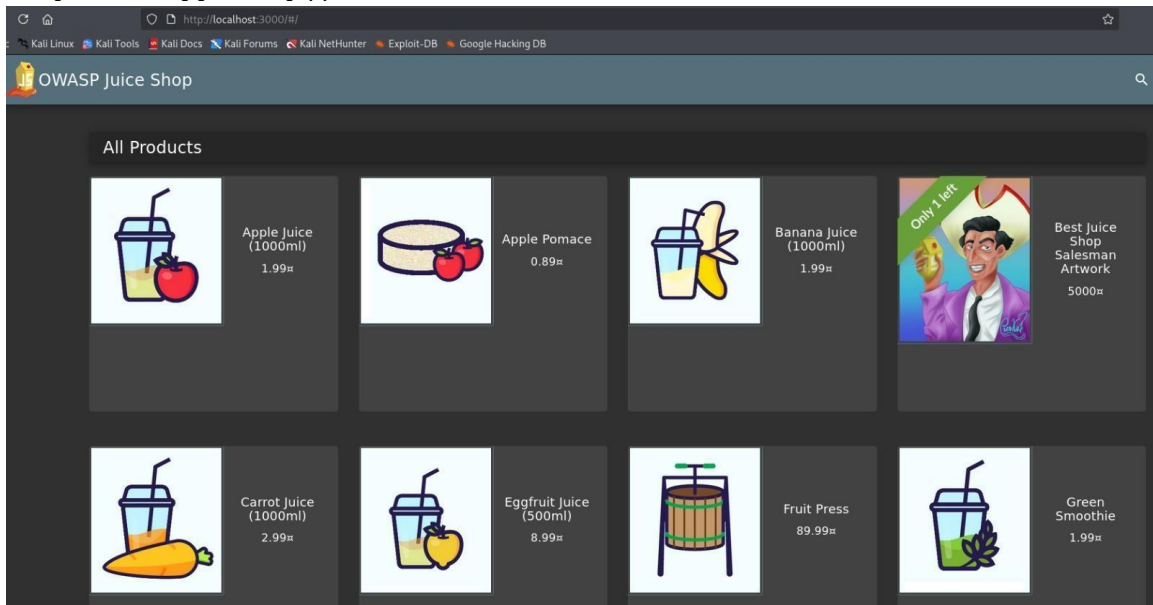
```
ahemii@ahemii:~$ sudo docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
35d697fe2738: Pull complete
bfb59b82a9b6: Pull complete
4eff9a62d888: Pull complete
62de241dac5f: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
5664b15f108b: Pull complete
045fc1c20da8: Pull complete
4aa0ea1413d3: Pull complete
da7816fa955e: Pull complete
ddf74a63f7d8: Pull complete
d00c3209d929: Pull complete
c058825cfcd6: Pull complete
7faf0cfa885c: Pull complete
5b14f6c9a813: Pull complete
33ce0b1d99fc: Pull complete
f45e0372ce60: Pull complete
46b46b3ee13c: Pull complete
e27f89e01: Pull complete
c223915a7ea9: Pull complete
Digest: sha256:c6f965f8929c2c43676e3ac55cd19d482c0084400195db07ed7513a04f3468b5
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
```

`docker run --rm -p 3000:3000 bkimminich/juice-shop`

```
ahemii@ahemii:~$ sudo docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
35d697fe2738: Pull complete
bfb59b82a9b6: Pull complete
4eff9a62d888: Pull complete
62de241dac5f: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
5664b15f108b: Pull complete
045fc1c20da8: Pull complete
4aa0ea1413d3: Pull complete
da7816fa955e: Pull complete
ddf74a63f7d8: Pull complete
d00c3209d929: Pull complete
c058825cfc6d: Pull complete
7faf0cfa885c: Pull complete
5b14f6c9a813: Pull complete
33ce0b1d99fc: Pull complete
f45e0372ce60: Pull complete
46b46b3ee13c: Pull complete
e27f89ebea01: Pull complete
c223915a7ea9: Pull complete
Digest: sha256:c6f965f8929c2c43676e3ac55cd19d482c0084400195db07ed7513a04f3468b5
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest
```

```
ahemii@ahemii:~$ sudo docker run --rm -p 3000:3000 bkimminich/juice-shop
info: Detected Node.js version v22.18.0 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file tutorial.js is present (OK)
info: Required file runtime.js is present (OK)
info: Required file vendor.js is present (OK)
info: Required file index.html is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
█
```

2. Open web app at <http://localhost:3000/>



3. Use Nmap to confirm service:

`nmap -p 3000 localhost`

```
ahemii@ahemii:~$ nmap localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-09 22:01 PKT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000090s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
3000/tcp  open  ppp

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds
```

Phase 1 — Local Reconnaissance

All Actions taken Now:

Browser exploration: login, product search, basket, feedback - Browser/Burp Suite proxy configured; all relevant requests captured and analyzed.

Connection Settings

×

Configure Proxy Access to the Internet

☐ No proxy

☐ Auto-detect proxy settings for this network

☐ Use system proxy settings

☒ Manual proxy configuration

HTTP Proxy

127.0.0.1

Port

8080

☒ Also use this proxy for HTTPS

HTTPS Proxy

127.0.0.1

Port

8080

SOCKS Host

Port

0

☐ SOCKS v4

☒ SOCKS v5

☐ Automatic proxy configuration URL

Reload

Cancel

OK

 OWASP Juice Shop

Login

Email*

user@gmail.com

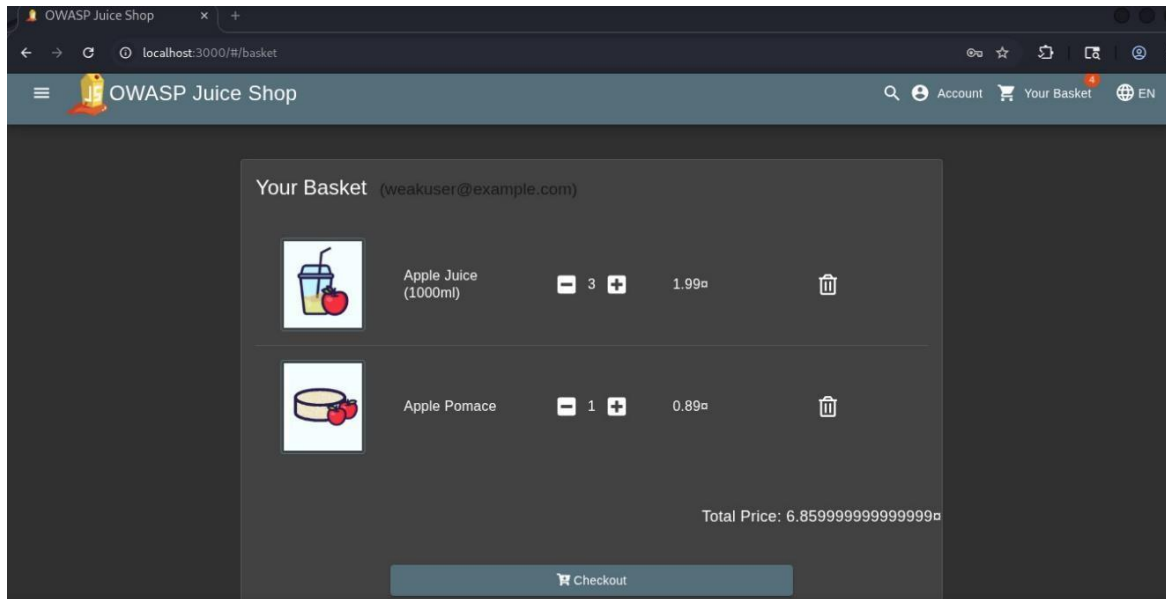
Password*

Forgot your password?

Log in

☐ Remember me

Not yet a customer?



Phase 2 — Vulnerability Findings (5 vulnerabilities)

Below are the documented vulnerabilities discovered during the audit. For each: Name & OWASP Category, Description, Steps to Reproduce, Payload used, Evidence (screenshot placeholder), Technical Explanation, and Remediation.

Vulnerability #1 — SQL Injection (A03: Injection)

Summary / Description:

A SQL-like injection was demonstrated against the login endpoint which allowed authentication bypass when the application concatenates user-controlled input into queries.

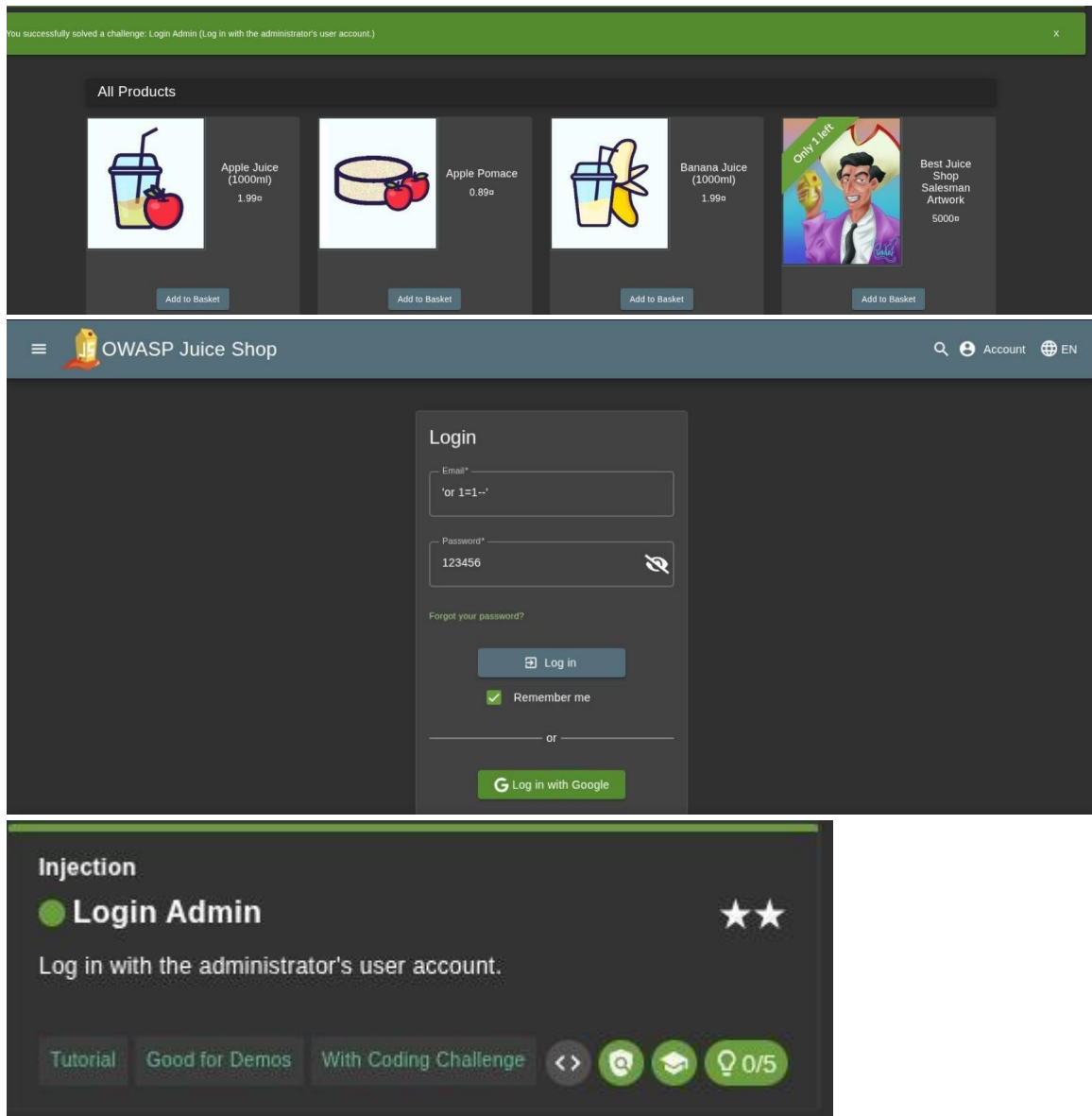
Steps to Reproduce :

1. Open the login page at <http://localhost:3000/#/login> (via Burp proxy).
2. Intercept the POST request to `/rest/user/login`.
3. Replace the 'email' field in the JSON body with the payload: `' OR 1=1--`
4. Forward the request from Burp Repeater. The server responded with a redirect to `/account` indicating the login was bypassed.

Payload used:

`' OR 1=1--`

Evidence :



Request snippet :

POST /rest/user/login HTTP/1.1

Host: localhost:3000

{"email":"' OR 1=1--","password":"1234"}

Remediation:

Use parameterized queries or prepared statements on the server side. Validate and sanitize inputs and apply least-privilege database accounts.

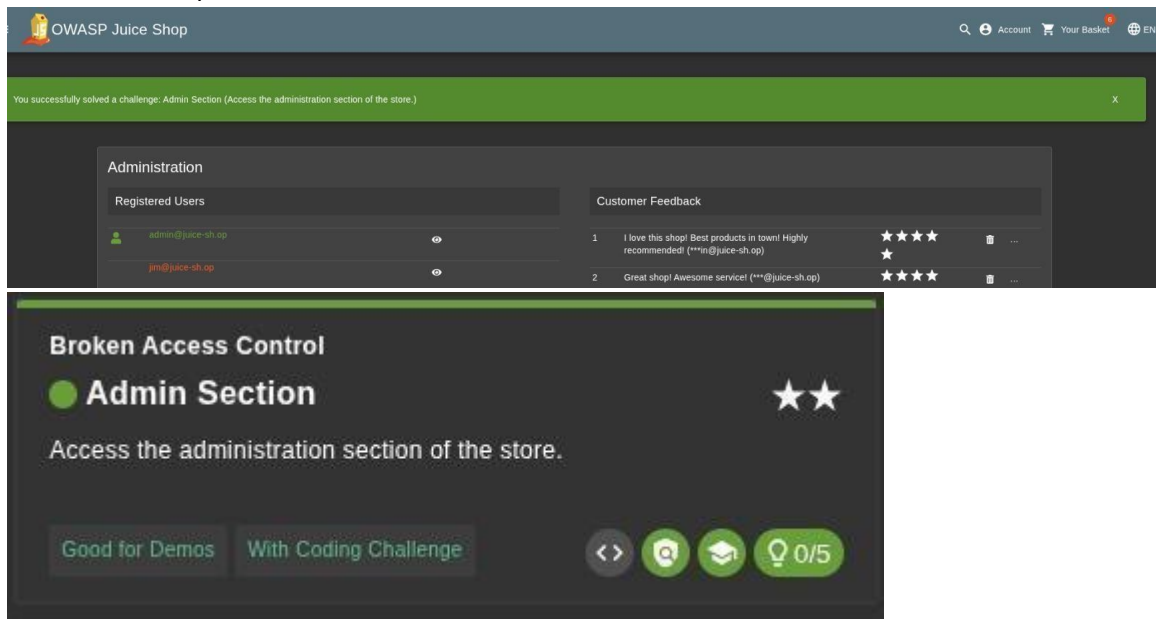
Vulnerability #2 — Broken Access Control (A01)

Summary / Description:

Accessing the admin panel without administration username or password

Steps to Reproduce :

1. Login as a normal user and capture a GET request that retrieves the admin
2. Send the request to Burp Repeater
3. Forward the modified request. And search in the domain Administration it will redirect to the admin page and you can easily bypass it by doing sql. The server returned HTTP/1.1 200 OK



Vulnerability #3 — Reflected Cross-Site Scripting (XSS)

Summary / Description:

The application reflects user-supplied input into HTML without proper encoding, enabling execution of JavaScript in victim browsers (reflected XSS).

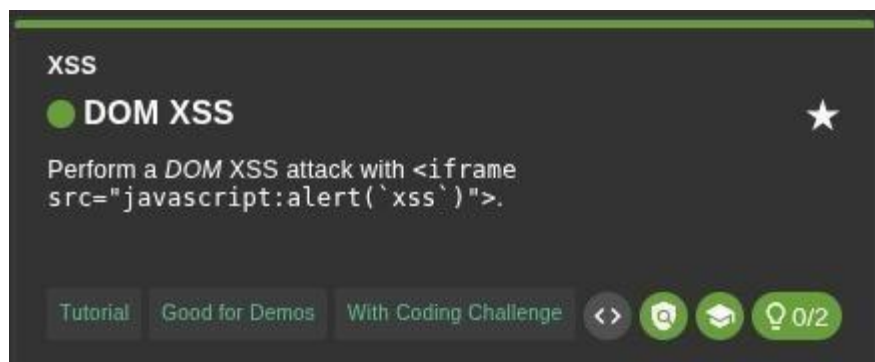
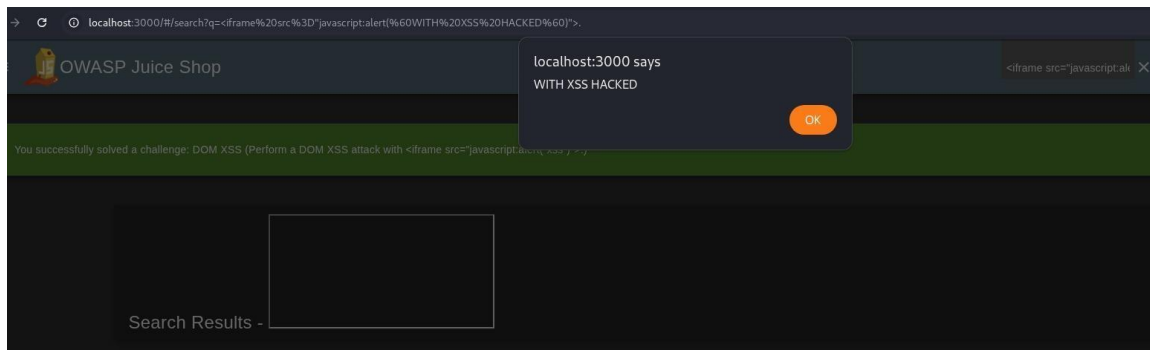
Steps to Reproduce :

1. In the search bar submit the payload:
`<iframe src="javascript:alert('xss')">`
2. The application reflected the input and the browser executed an alert dialog with 'xss'.

Payload used:

`<iframe src="javascript:alert('xss')">`

Evidence :



Remediation:

Apply contextual output encoding for HTML, use framework-provided escaping functions, and implement a Content Security Policy (CSP).

Vulnerability #4 — Security Misconfiguration (A05)

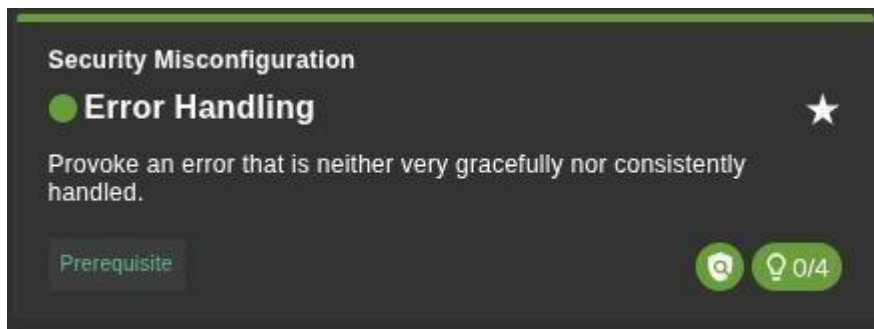
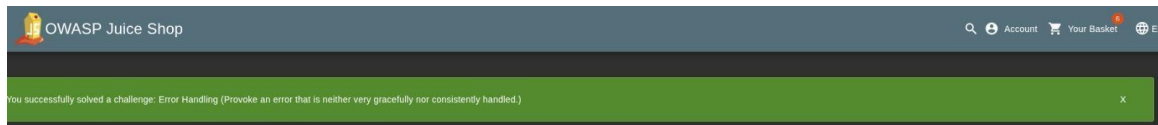
Summary / Description:

Malformed requests caused the server to disclose verbose error messages and stack traces, revealing internal file paths and framework versions.

Steps to Reproduce :

1. Using Burp Repeater, send a malformed USER payload to an API endpoint.
2. The server returned an error response that included a Node.js/Express stack trace and file path information.

Evidence :



Remediation:

Disable debug/stack traces in production, return generic error messages to clients, and log detailed errors only server-side.

Vulnerability #5 — Reflected Cross-Site Scripting (XSS)

Summary / Description:

The application reflects user-supplied input into HTML without proper encoding, enabling execution of JavaScript in victim browsers (reflected XSS).

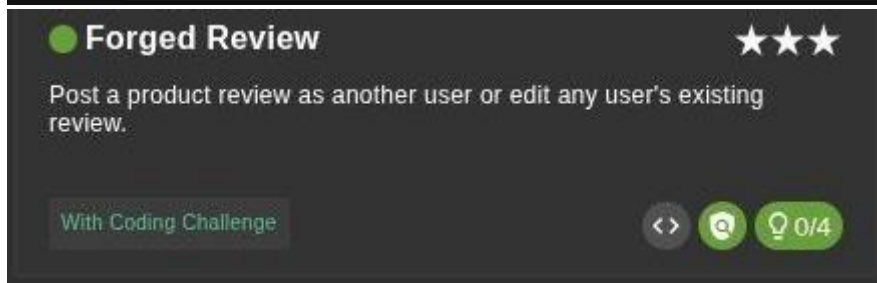
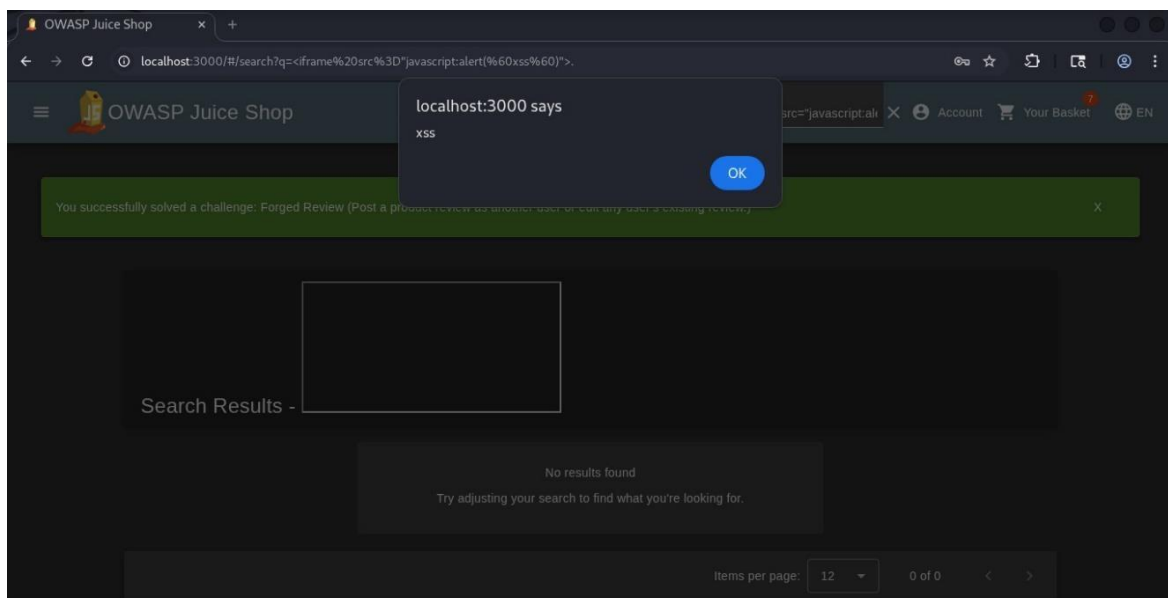
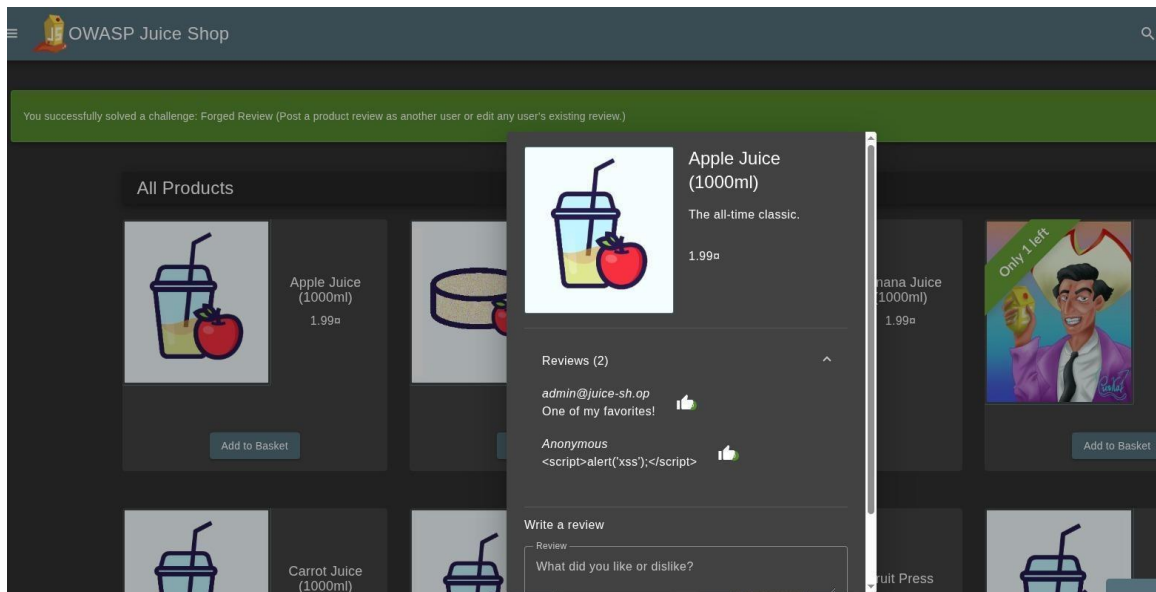
Steps to Reproduce :

1. In the review/comment section submit the payload:
`<script>alert('xss');</script>`
2. The application reflected the input and the browser executed an alert dialog with 'xss'.

Payload used:

`<script>alert('xss');</script>`

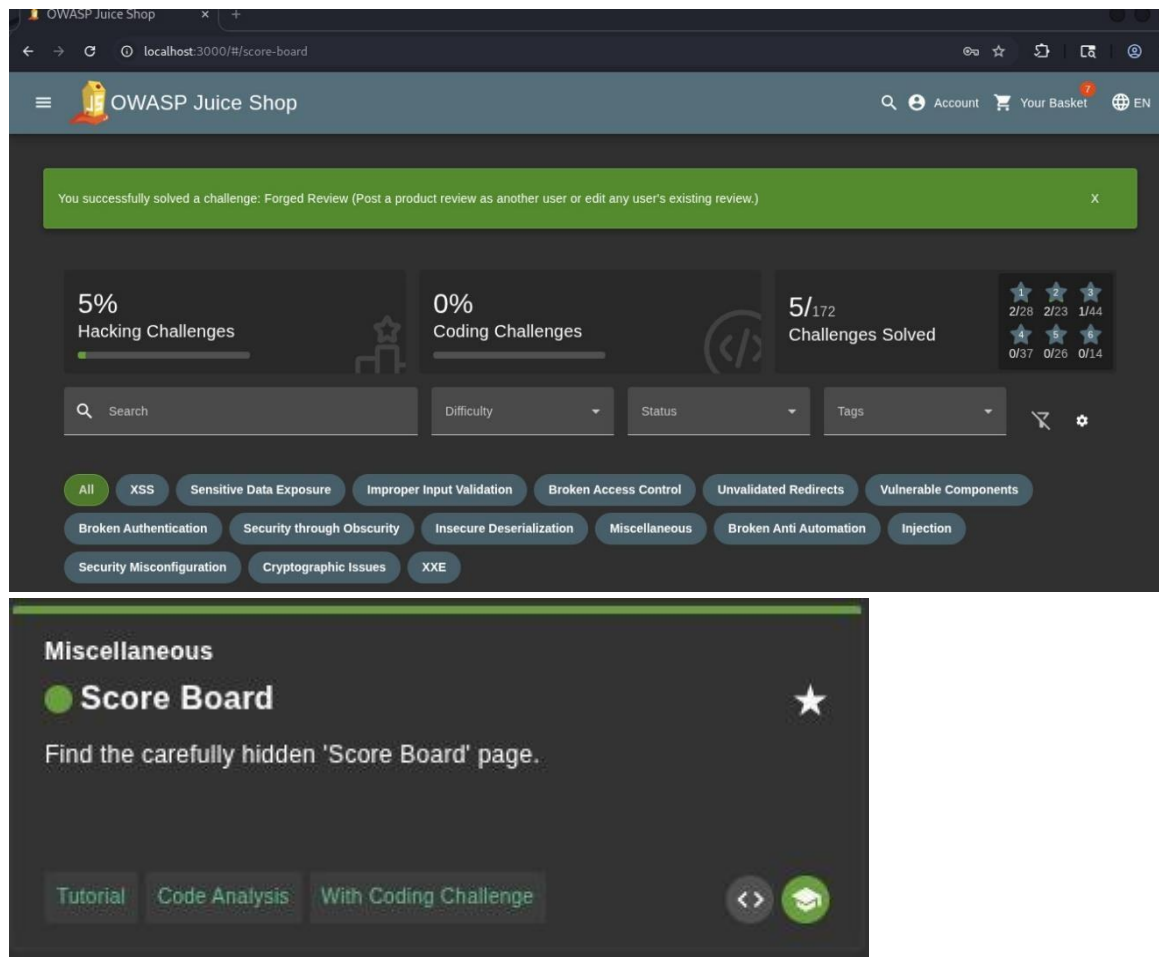
Evidence :



Remediation:

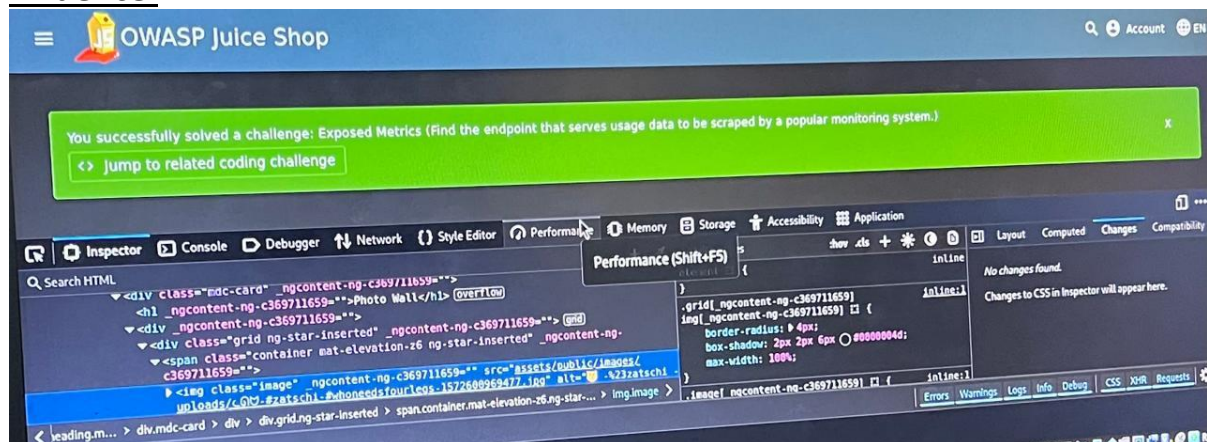
Apply contextual output encoding for HTML, use framework-provided escaping functions, and implement a Content Security Policy (CSP).

Score Board (Challenge Proof)



Vulnerability #6 - XML External Entity (XXE)

Evidence:

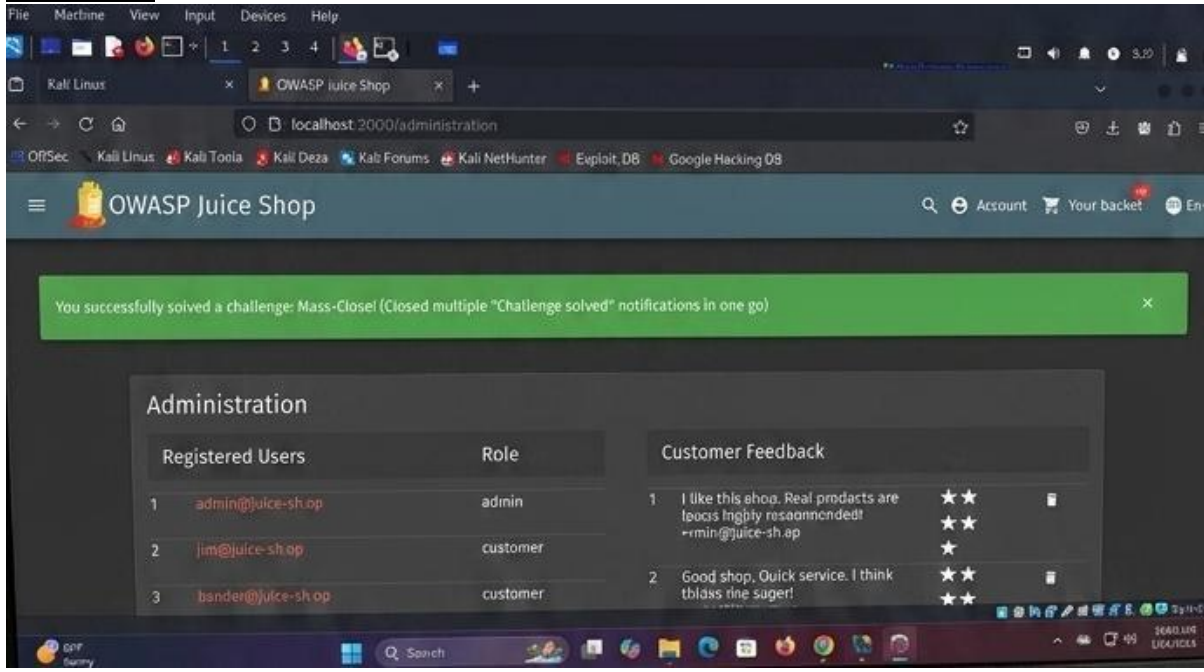


Description:

XML parser allows external entity injection → file disclosure.

Vulnerability #7 - Invalidated Redirects

Evidence:

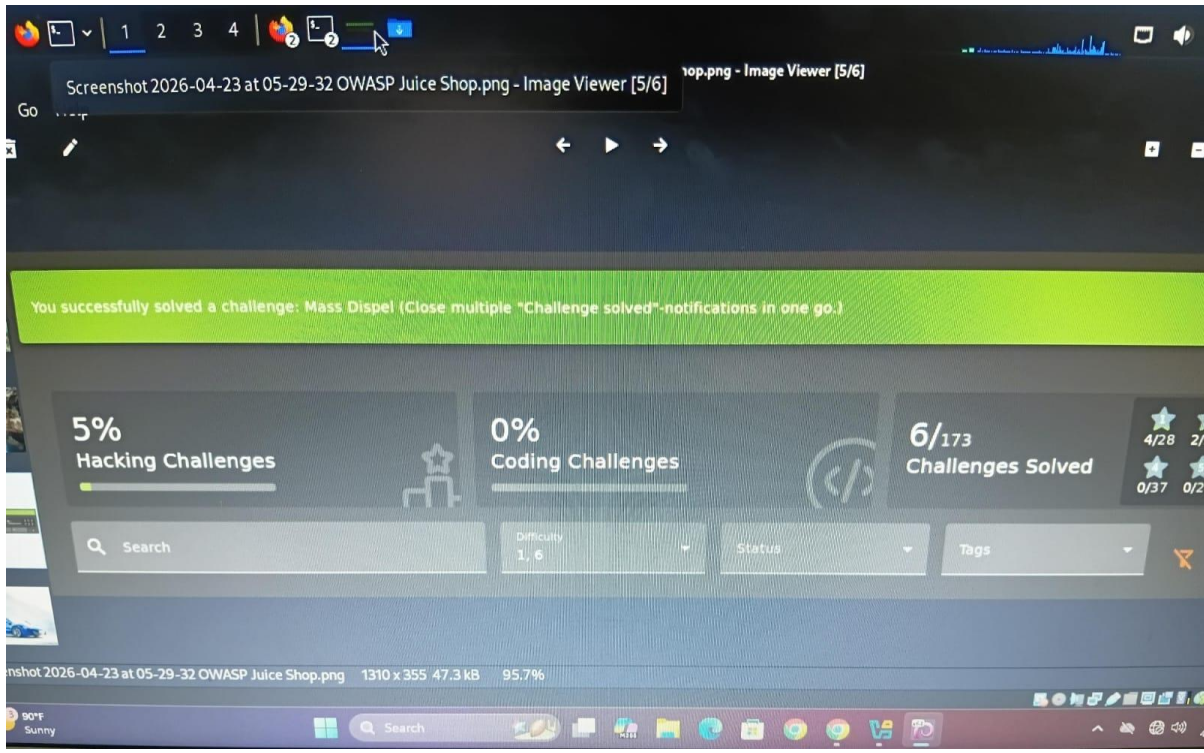


Description:

“Allow list Bypass”, “Outdated Allowlist”
Users redirected to malicious websites.

Vulnerability # 8 - Insecure Deserialization

Evidence:



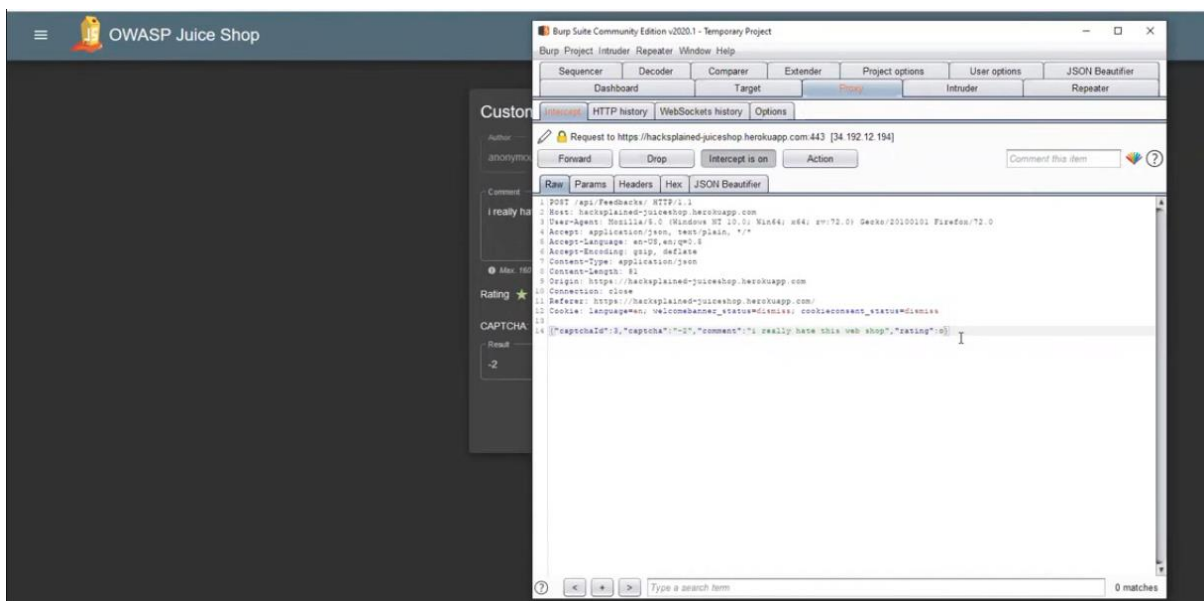
Description:

Description: "Memory Bomb", "RCE DoS" challenges

Malicious serialized objects can cause DoS or code execution.

Vulnerability #9 - Insecure Direct Object References (IDOR)

Evidence:



Customer Feedback

Thank you for your feedback

Author

Comment

Max. 190 characters 5/190

Rating ★ ★ ★ ★ ★

CAPTCHA: What is 10-9-3 ?

Result

Submit

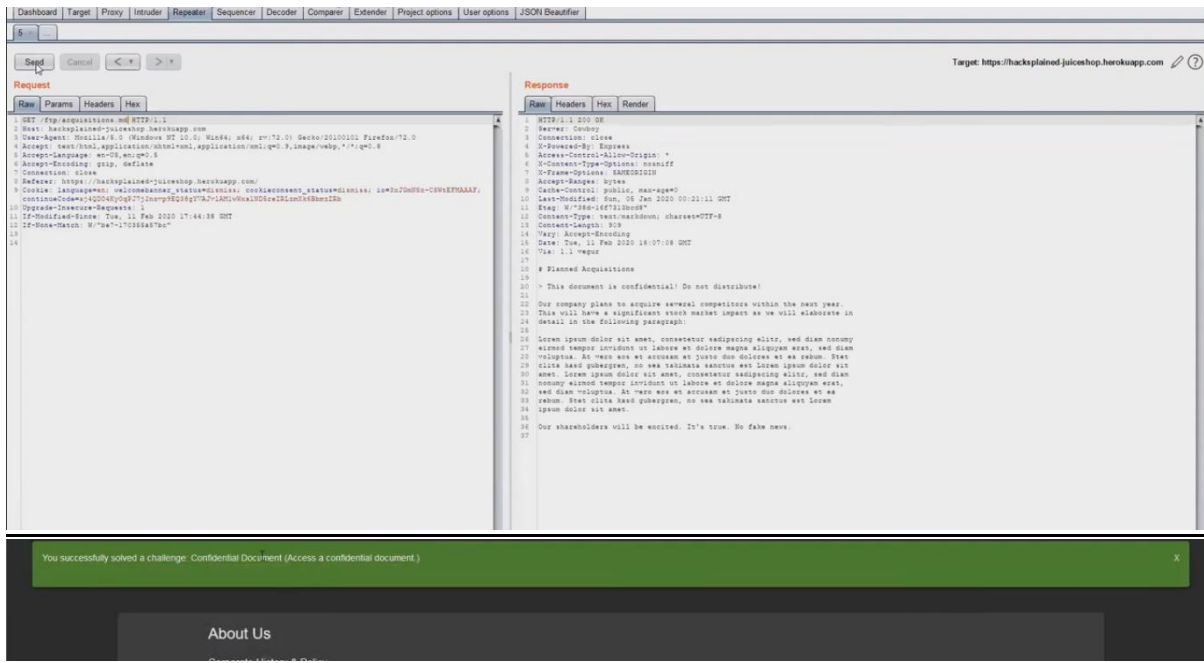
Description:

Taken out the insecurity of the customers feedback!

Access other users' data by changing IDs.

Vulnerability #10 - Using Components with Known Vulnerabilities

Evidence:

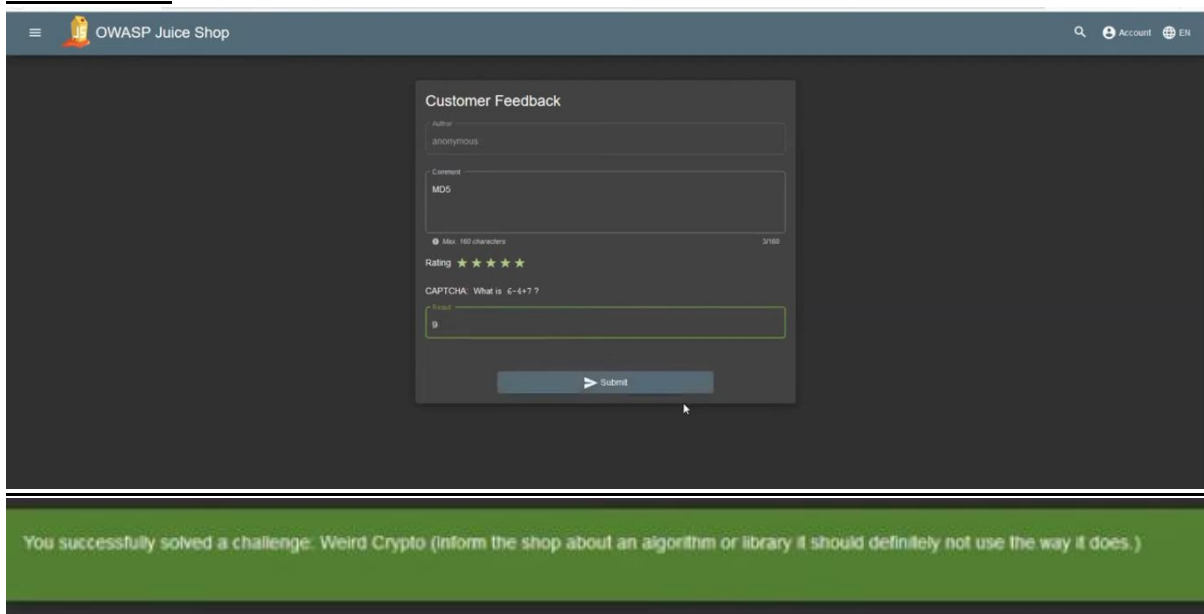


Description:

Outdated libraries and dependencies.

Vulnerability #11 – Crypto Graphical Issue

Evidence:



Description:

Cryptographic Issues:

Cryptographic issues occur when an application uses weak, outdated, or improperly implemented encryption methods.

Sensitive data like passwords or tokens may be stored in plain text or with weak hashing

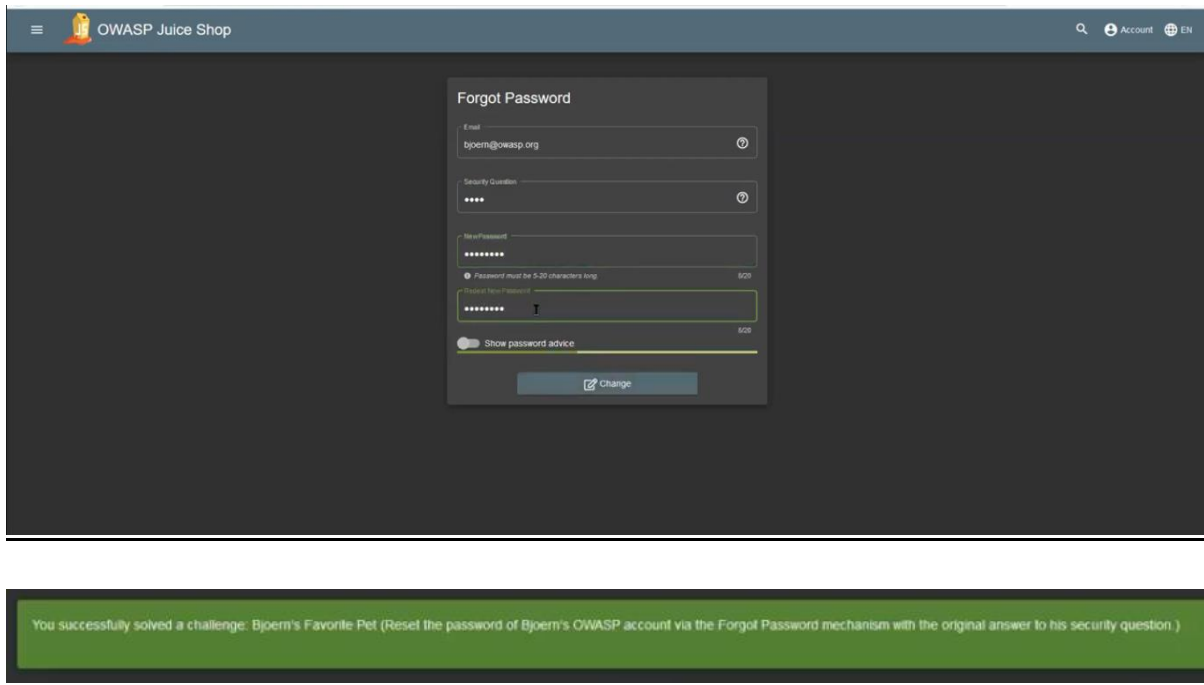
algorithms.

Poor key management (hardcoded or exposed keys) can allow attackers to decrypt protected information.

This can lead to data breaches, account compromise, and loss of confidentiality.

Vulnerability #12 – Björn Private Pet

Evidence:

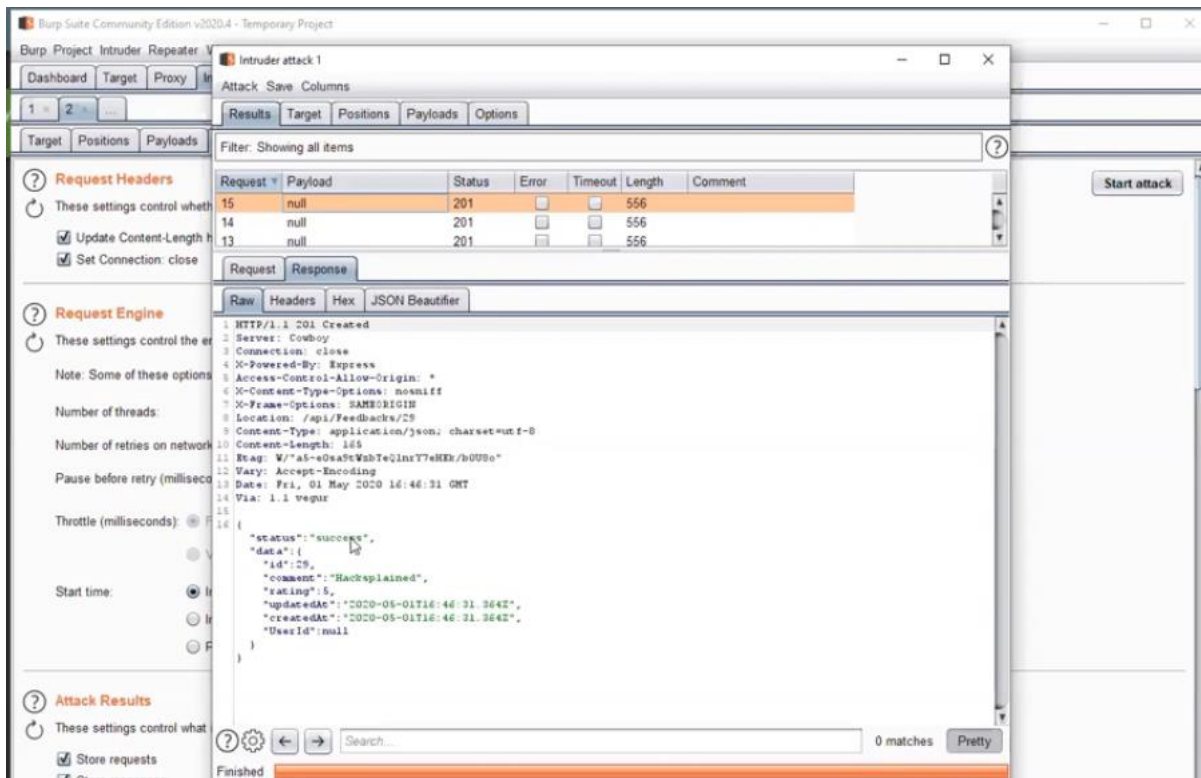


Description:

It refers to a hidden resource belonging to the fictional character Björn, which should not be publicly accessible. Due to improper access restrictions, attackers can discover and access this private content without authorization. This demonstrates how sensitive files or endpoints can be exposed through weak security controls. The vulnerability highlights the importance of proper authentication and restricting access to private data.

Vulnerability #13 – Captcha Bypass

Evidence:

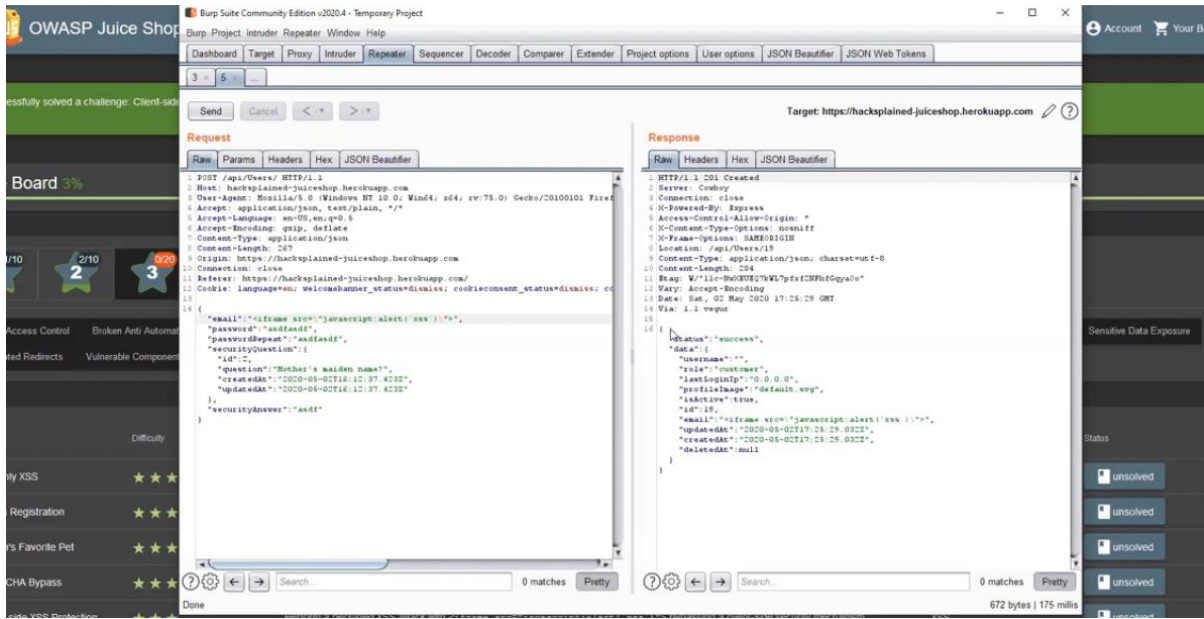


Description:

CAPTCHA bypass occurs when an application fails to properly validate whether a user has solved the CAPTCHA challenge. Attackers can skip or manipulate the verification process using automated scripts or by modifying client-side requests. This allows bots to perform actions like brute-force attacks, spam submissions, or fake account creation. It highlights weak server-side validation and over-reliance on client-side security controls.

Vulnerability #14 – Client Side XSS

Evidence:

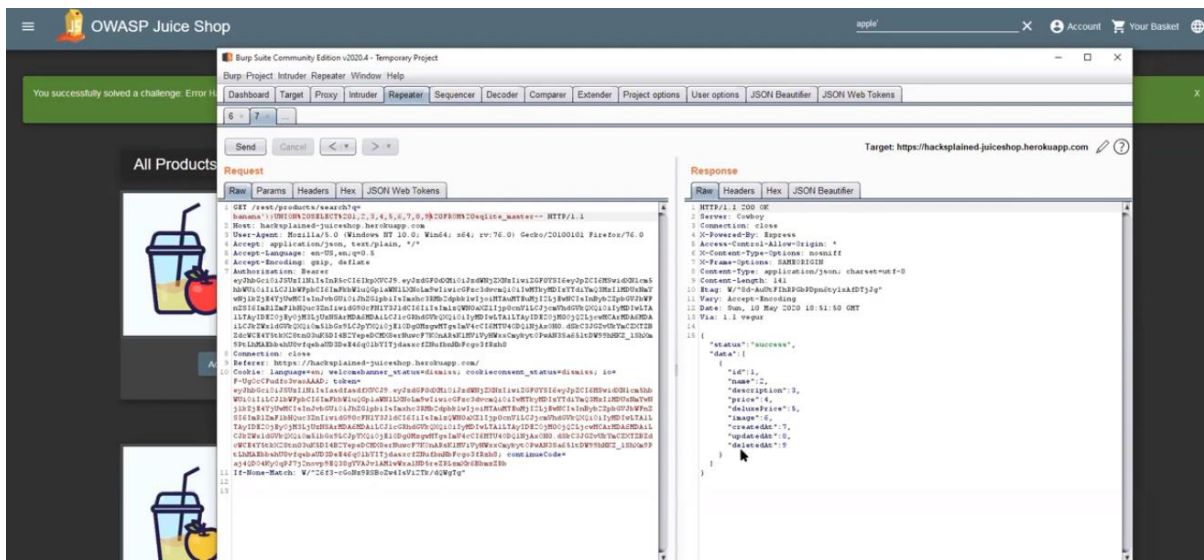


Description:

Client-side XSS occurs when malicious scripts are executed in the user's browser due to insecure handling of input in front-end code. It typically arises from unsafe JavaScript operations like directly inserting user input into the DOM (e.g., innerHTML). Attackers can inject scripts that steal cookies, session tokens, or manipulate page content. This vulnerability highlights the need for proper input sanitization and secure DOM manipulation practices.

Vulnerability #15 – Database Schema

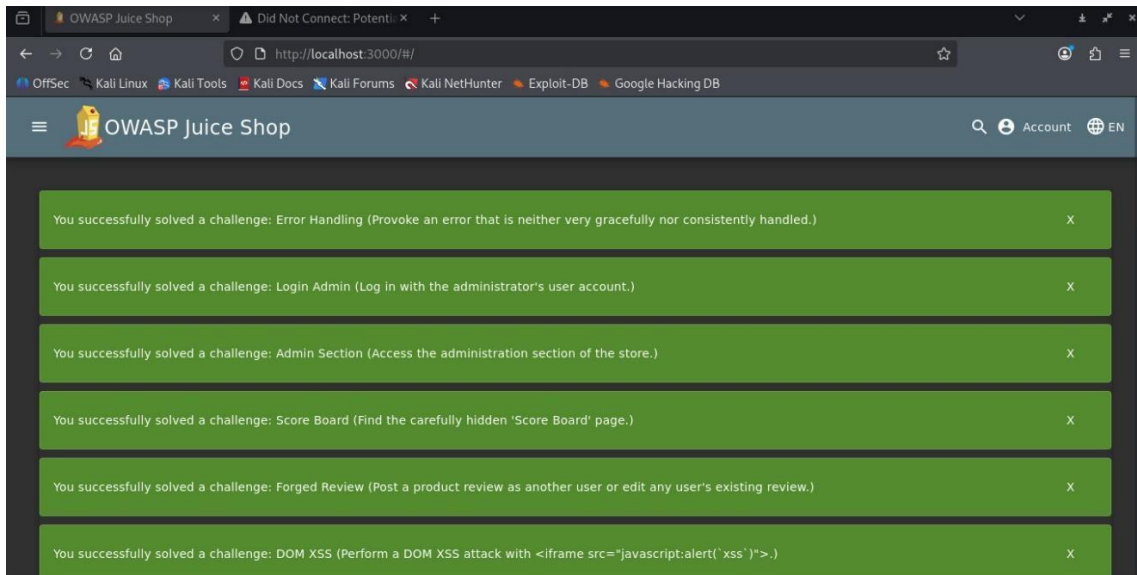
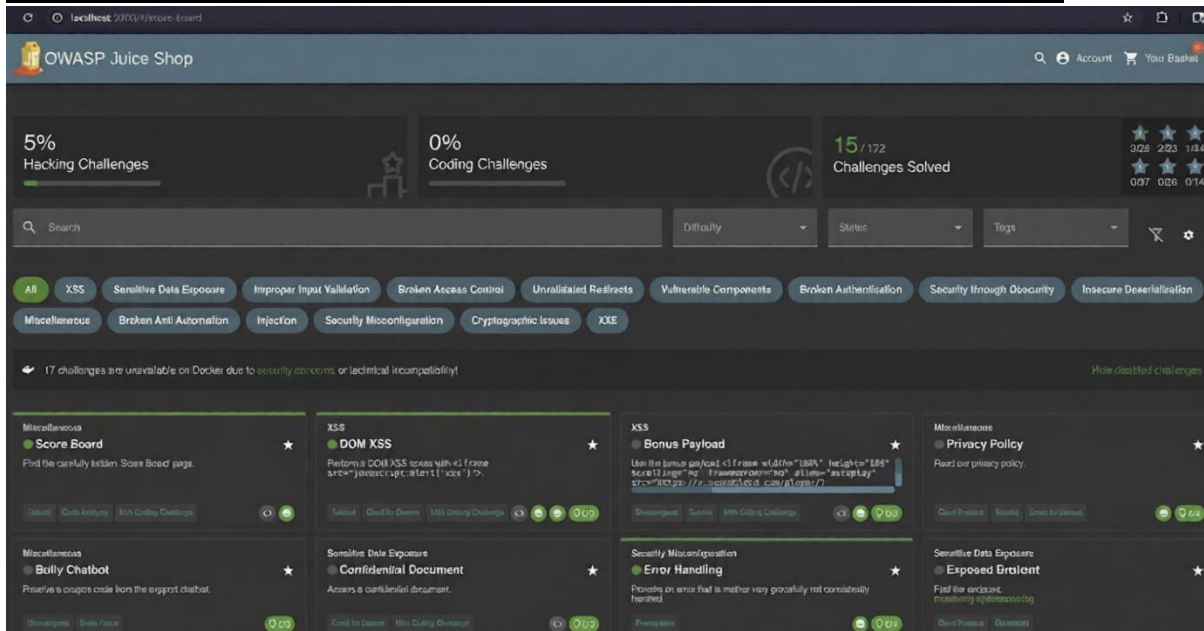
Evidence:

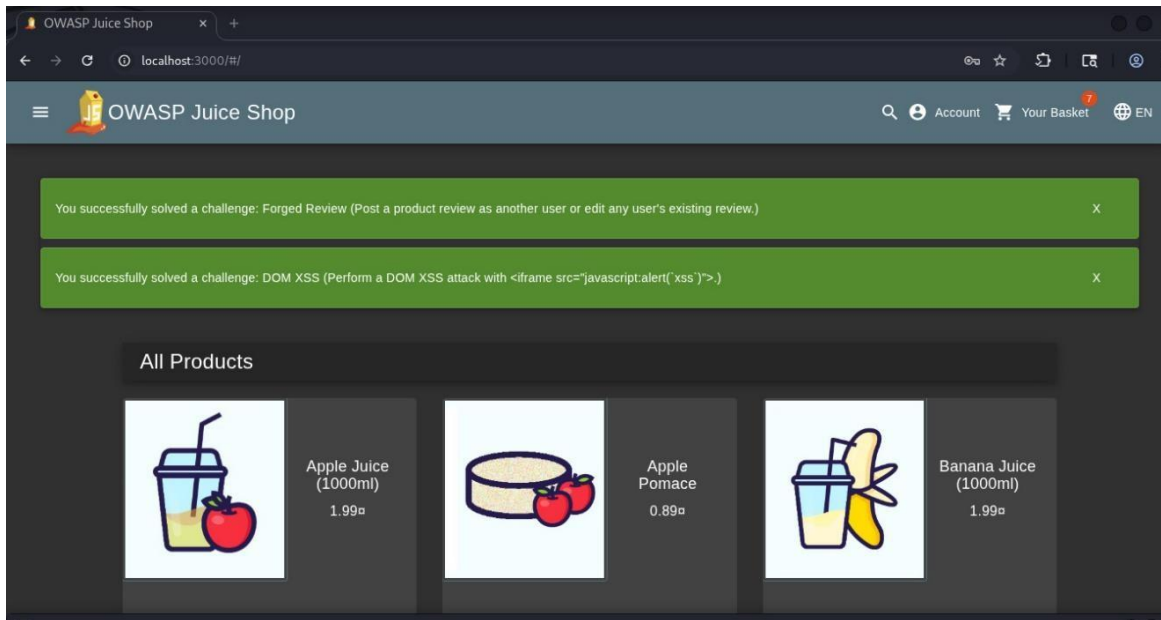


Description:

A database schema defines the structure and organization of data within a database system. It includes tables, columns, data types, relationships, and constraints that govern how data is stored. Schemas help maintain data integrity, consistency, and efficient querying. They act as a blueprint for how an application interacts with its database.

15 VULNERABILITIES HAVE BEEN EXPLOITED INCLUDING SCORE-BOARD





7. Burp Suite

```
ahemi@ahemii:~$ burpsuite
[warning] /usr/bin/burpsuite: No JAVA_CMD set for run_java, falling back to JAVA_CMD = java
Your JRE appears to be version 21.0.9 from Debian
Burp has not been fully tested on this platform and you may experience problems.
Deleting temporary files - please wait ... done.
```

```
Info: Detected CPU x64 (OK)
Info: Configuration default validated (OK)
Info: Entity models 20 of 20 are initialized (OK)
Info: Required file server.js is present (OK)
Info: Required file styles.css is present (OK)
Info: Required file main.js is present (OK)
Info: Required file tutorial.js is present (OK)
Info: Required file runtime.js is present (OK)
Info: Required file vendor.js is present (OK)
Info: Required file index.html is present (OK)
Info: Port 3000 is available (OK)
Info: Chatbot training data botDefaultTrainingData.json validated (OK)
Info: Domain https://www.alchemy.com/ is reachable (OK)
Info: Server listening on port 3000
Error
  at Database.<anonymous> (/juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:185:27)
  at /juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:183:50
  at new Promise (<anonymous>)
  at Query.run (/juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:183:12)
  at /juice-shop/node_modules/sequelize/lib/sequelize.js:315:28
  at async SQLiteQueryInterface.insert (/juice-shop/node_modules/sequelize/lib/dialects/abstract/query-interface.js:308:21)
  at async BasketItem.save (/juice-shop/node_modules/sequelize/lib/model.js:2490:35)
Info: Solved 1-star errorHandlingChallenge (Error Handling)
Info: Cheat score for trivial errorHandlingChallenge solved in 105min (expected ~0min) with hints allowed: 0
Error
  at Database.<anonymous> (/juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:185:27)
  at /juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:183:50
  at new Promise (<anonymous>)
  at Query.run (/juice-shop/node_modules/sequelize/lib/dialects/sqlite/query.js:183:12)
  at /juice-shop/node_modules/sequelize/lib/sequelize.js:315:28
  at async SQLiteQueryInterface.insert (/juice-shop/node_modules/sequelize/lib/dialects/abstract/query-interface.js:308:21)
  at async BasketItem.save (/juice-shop/node_modules/sequelize/lib/model.js:2490:35)
Info: Solved 2-star loginAdminChallenge (Login Admin)
Info: Cheat score for tutorial loginAdminChallenge solved in 11min (expected ~2min) with hints allowed: 0
Info: Solved 2-star adminSectionChallenge (Admin Section)
Info: Cheat score for tutorial adminSectionChallenge solved in 9min (expected ~2min) with hints allowed: 0
Info: Solved 1-star scoreBoardChallenge (Score Board)
Info: Cheat score for tutorial scoreBoardChallenge solved in 5min (expected ~1min) with hints allowed: 0
Info: Solved 3-star forgedReviewChallenge (Forged Review)
Info: Cheat score for forgedReviewChallenge solved in 15min (expected ~6min) with hints allowed: 0
Info: Solved 1-star localXssChallenge (DOM XSS)
Info: Cheat score for tutorial localXssChallenge solved in 11min (expected ~1min) with hints allowed: 0
```

Tools Used

- Kali Linux
- Docker
- Burp Suite (Community Edition)
- Nmap
- Mozilla Firefox

Conclusion:

The security audit of the OWASP Juice Shop web application revealed multiple critical and medium-level vulnerabilities, including SQL Injection, Broken Access Control, Cross-Site Scripting (XSS), and Security Misconfiguration. These weaknesses allowed authentication bypass, unauthorized admin access, JavaScript code execution in users' browsers, and leakage of internal server information. The findings confirm that the application lacks proper input validation, output encoding, and secure configuration, making it vulnerable to data compromise and unauthorized actions.

Recommendations:

It is strongly recommended to fix these vulnerabilities by implementing parameterized queries to prevent SQL injection, enforcing strict server-side access control for all sensitive routes, and applying proper output encoding and a strong Content Security Policy to mitigate XSS. Additionally, disable verbose error messages in production, limit database and container privileges, and apply secure headers to harden the deployment. After these fixes, re-test the application using Burp Suite and automated scanners to ensure no critical vulnerabilities remain.